

МИНИСТЕРСТВО ОБРАЗОВАНИЯ ОРЕНБУРГСКОЙ ОБЛАСТИ
Государственное автономное профессиональное образовательное
учреждение
«ОРЕНБУРГСКИЙ КОЛЛЕДЖ ЭКОНОМИКИ И ИНФОРМАТИКИ»
(ГАПОУ ОКЭИ)

Контрольная точка 5
ОКЭИ 09.02.07 4025 19 ПЗ

Тема: «Создание тестовой документации»

Выполнил: Сулейманов Эмиль Ильдарович

Оренбург 2026

Содержание

1 Ссылка на гит.....	3
2 Требования к тестированию.....	4
2.1 Введение.....	4
2.3 Нефункциональные требования.....	5
2.4 Риски.....	5
2.5 Критерии начала тестирования.....	5
2.6 Критерии завершения тестирования	6
3 Тест-план.....	7
4 Тест-кейсы.....	8
5 Чек-листы	12

					ОКЭИ 09.02.07 4025 19 П					
Изм.	Лист	№ докум.	Подп.	дата						
Разраб.	Сулейманов Э.И.				Практическая работа			Лит.	Лист	Листов
	Краснов Н.А.							У		
								Отделение информационных систем		

1 Ссылка на гит

<https://github.com/beaaaad1/FinancialCalculator>

					ОКЭИ 09.02.07 4025 19 П	Лист
Изм.	Лист	№ докум.	Подп.	Дата		3

2 Требования к тестированию

2.1 Введение

Данный документ описывает требования к тестированию консольного приложения «Финансовый калькулятор». Приложение реализовано на языке C# и предоставляет пользователю три финансовых инструмента: кредитный калькулятор, конвертер валют и калькулятор вкладов. Цель документа — зафиксировать, что именно должно быть проверено в ходе тестирования, какие характеристики системы являются критически важными и какие риски могут повлиять на качество продукта.

2.2 Функциональные требования

Кредитный калькулятор

Модуль должен принимать три входных параметра: сумму кредита в рублях, срок в месяцах и процентную ставку годовых. На основе этих данных программа рассчитывает аннуитетный ежемесячный платёж по формуле:

Платёж = Сумма $\times (r \times (1+r)^n) / ((1+r)^n - 1)$, где r — месячная ставка, n — срок в месяцах.

Если процентная ставка равна нулю, расчёт упрощается: ежемесячный платёж = сумма / срок. Помимо ежемесячного платежа, модуль должен выводить общую сумму выплат и переплату по кредиту. Все входные данные должны быть строго положительными числами, иначе программа должна запросить повторный ввод.

Конвертер валют

Модуль выполняет конвертацию между тремя валютами: RUB, USD и EUR. Курсы зафиксированы в коде и не изменяются в процессе работы: USD→RUB = 90.0, EUR→RUB = 98.5, EUR→USD = 1.09. Конвертация между любыми двумя валютами должна выполняться через рублёвый эквивалент. Если пользователь вводит одинаковые исходную и целевую валюту, результат должен совпадать с введённой суммой. Регистр ввода валюты не должен влиять на результат — то есть «usd», «USD» и «Usd» должны обрабатываться одинаково.

Калькулятор вкладов

Модуль рассчитывает доход по вкладу двумя способами в зависимости от выбранного типа. При вкладе без капитализации доход рассчитывается по формуле: Доход = Сумма $\times$ Ставка $\times$ Срок / 12 / 100. При вкладе с капитализацией итоговая сумма рассчитывается по формуле: Итог = Сумма $\times (1 + \text{Ставка}/100/12)^{\text{Срок}}$. В обоих случаях программа выводит доход по вкладу и итоговую сумму.

Навигация и интерфейс

При запуске программа должна отображать главное меню с четырьмя пунктами. Выбор пунктов 1–3 открывает соответствующий модуль, пункт 4 завершает работу программы. При вводе любого другого значения программа должна вывести сообщение об ошибке и снова показать меню, не завершая работу.

2.3 Нефункциональные требования

Помимо функциональной корректности, к приложению предъявляется ряд нефункциональных требований. Время отклика на любую операцию не должно превышать 2 секунд на стандартном оборудовании. Интерфейс программы должен быть полностью на русском языке, включая все сообщения об ошибках и подсказки. Приложение должно корректно работать на платформе .NET 6.0 и выше под управлением Windows 10/11. Результаты всех расчётов должны округляться до двух знаков после запятой. При вводе некорректных данных программа не должна аварийно завершаться — все исключения должны быть обработаны.

2.4 Риски

Наиболее серьёзным техническим риском являются ошибки в математических формулах. Неверная реализация аннуитетного расчёта или формулы капитализации приведёт к тому, что все результаты будут некорректны. Связанный риск — ошибки округления: при работе с числами с плавающей точкой результат может незначительно отличаться от эталонного значения, что важно учитывать при сравнении ожидаемых и фактических результатов в тест-кейсах.

Отдельный риск представляет формат ввода чисел. В зависимости от региональных настроек системы программа может не распознавать дробные числа, введённые через запятую вместо точки (или наоборот). Также существует риск некорректной обработки граничных значений — нулей, очень больших чисел или отрицательных значений.

2.5 Критерии начала тестирования

Тестирование может быть начато только при выполнении следующих условий: исходный код успешно скомпилирован без ошибок, программа запускается и отображает главное меню, на машине тестировщика установлена среда выполнения .NET нужной версии.

					ОКЭИ 09.02.07 4025 19 П	Лист
Изм.	Лист	№ докум.	Подп.	Дата		5

2.6 Критерии завершения тестирования

Тестирование считается завершённым, когда выполнены все тест-кейсы высокого приоритета, не менее 80% тест-кейсов среднего приоритета, все чек-листы пройдены, а все найденные критические и серьёзные дефекты исправлены и повторно проверены.

3 Тест-план

Приложение: Финансовый калькулятор v1.0

Язык разработки: C# (.NET 6.0+)

Тип приложения: Консольное приложение Windows

В объём тестирования включены все три модуля приложения. В кредитном калькуляторе проверяется корректность аннуитетного расчёта, обработка нулевой ставки и валидация входных параметров. В конвертере валют проверяются все направления конвертации между RUB, USD и EUR, корректность применения фиксированных курсов и обработка некорректных названий валют. В калькуляторе вкладов проверяются обе формулы расчёта — с капитализацией и без — а также валидация входных данных. Помимо модулей, тестируется навигация по главному меню и обработка неверного ввода. Из тестирования исключены интеграция с внешними системами, тестирование безопасности, кроссплатформенная совместимость с Linux и macOS, а также нагрузочное тестирование.

В рамках данного тест-плана применяется несколько видов тестирования. Основным является функциональное тестирование — проверка соответствия поведения программы задокументированным требованиям. Дополнительно проводится тестирование граничных значений, негативное тестирование с намеренно некорректными данными, smoke-тестирование перед каждым полным циклом и регрессионное тестирование после исправления дефектов.

Работы разбиты на шесть этапов общей продолжительностью 1 рабочий день. Сначала идет подготовка: настройка окружения, изучение требований и подготовку тестовых данных. Затем smoke-тестирование — быстрая проверка работоспособности основных функций. Далее функциональное тестирование всех трёх модулей с фиксацией дефектов. Затем негативное тестирование и проверку граничных значений. После всего исправление дефектов.

Тестирование проводится на операционной системе Windows 10 или Windows 11 со средой выполнения .NET 6.0 и выше, в среде разработки Visual Studio 2022. Запуск приложения осуществляется через CMD, PowerShell или Windows Terminal.

Тестирование считается успешно завершённым при выполнении следующих критериев приёмки. Расчёт аннуитетного платежа должен совпадать с эталонным значением с точностью до ± 0.01 руб. Результат конвертации валют должен соответствовать фиксированным курсам с той же точностью. Обе формулы калькулятора вкладов должны давать корректный результат. На уровне всего приложения: выполнено 100% тест-кейсов высокого приоритета, не менее 80% тест-кейсов среднего приоритета, отсутствуют незакрытые дефекты уровня «Критический» и «Серьёзный», все чек-листы пройдены с результатом PASSED.

					ОКЭИ 09.02.07 4025 19 П	Лист
Изм.	Лист	№ докум.	Подп.	Дата		7

4 Тест-кейсы

ID	Название	Описание	Шаги	Ожидаемый результат	Фактический результат
1	Расчёт аннуитетного платежа	Проверка корректности расчёта при валидных данных.	1. Сумма: 100000 2. Срок: 12 3. Ставка: 10%	Платёж: 8791.59 руб. Сумма: 105499.08 руб. Переплата: 5499.08 руб.	Успешно
2	Расчёт при нулевой ставке	Проверка упрощённой формулы когда ставка = 0.	1. Сумма: 120000 2. Срок: 12 3. Ставка: 0	Платёж: 10000.00 руб. Переплата: 0.00 руб.	Успешно – в коде есть отдельная ветка if (rate == 0)
3	Отрицательная сумма кредита	Проверка валидации при вводе отрицательного числа.	1. Сумма: -5000	Сообщение об ошибке, запрос повторного ввода	Успешно – метод GetValidDoubleInput имеет параметр minValue: 0
4	Текст вместо числа в сумме	Проверка валидации при вводе нечислового значения.	1. Сумма: abc	Сообщение «Неверный формат», запрос повторного ввода	Успешно – double.TryParse отклоняет нечисловые значения
5	Срок кредита равен нулю	Проверка валидации при сроке = 0.	1. Сумма: 100000 2. Срок: 0	Сообщение об ошибке, запрос повторного ввода	Успешно

6	Расчёт с большими значениями	Проверка расчёта при максимальных значениях.	1. Сумма: 1000000 0 2. Срок: 360 3. Ставка: 12	Программа выводит корректные числа без ошибок и зависаний	Успешно
7	Конвертация USD в RUB	Проверка курса USD→RUB = 90.0.	1. Исходная: USD 2. Целевая: RUB 3. Сумма: 100	9000.00 RUB	Успешно
8	Конвертация EUR в RUB	Проверка курса EUR→RUB = 98.5.	1. Исходная: EUR 2. Целевая: RUB 3. Сумма: 100	9850.00 RUB	Успешно
9	Конвертация RUB в USD	Проверка обратной конвертации.	1. Исходная: RUB 2. Целевая: USD 3. Сумма: 9000	100.00 USD	Успешно
10	Конвертация EUR в USD	Проверка курса EUR→USD = 1.09.	1. Исходная: EUR 2. Целевая: USD 3. Сумма: 100	109.00 USD	Ошибка — в коде конвертация идёт через RUB, а не напрямую по курсу 1.09. Результат будет $98.5/90.0 \times 100$

					= 109.44 USD вместо 109.00
11	Конвертация в ту же валюту	Сумма не должна изменяться.	1. Исходная: USD 2. Целевая: USD 3. Сумма: 250	250.00 USD	Успешно – в коде есть проверка if (from == to) return amount
12	Несуществующая валюта	Проверка обработки неверного кода валюты.	1. Исходная валюта: GBP	Сообщение «Неверно указана валюта», программа не падает	Успешно – метод IsValidCurrency проверяет допустимые значения
13	Ввод валюты в нижнем регистре	Регистр не должен влиять на результат.	1. Исходная: usd 2. Целевая: rub 3. Сумма: 100	9000.00 RUB	Успешно – в коде применяется .ToUpper().Trim()
14	Вклад без капитализации	Проверка формулы: $\text{Сумма} \times \text{Ставка} \times \text{Срок} / 12 / 100$.	1. Сумма: 100000 2. Срок: 12 3. Ставка: 10 4. Тип: 1	Доход: 10000.00 руб. Итог: 110000.00 руб.	Успешно
15	Вклад с капитализацией	Проверка формулы: $\text{Сумма} \times (1 + \text{Ставка} / 100 / 12)^{\text{Срок}}$.	1. Сумма: 100000 2. Срок: 12 3. Ставка: 10 4. Тип: 2	Доход: 10471.31 руб. Итог: 110471.31 руб.	Успешно
16	Неверный тип вклада	Проверка обработки значения вне диапазона 1-2.	1. Тип вклада: 5	Сообщение «Неверно	Успешно – в коде есть ветка else с

				ый выбор типа вклада», програм ма не падает	throw new Exception
17	Навигация по всем пунктам меню	Проверка перехода в каждый модуль.	1. Ввести 1 — вернуть ся 2. Ввести 2 — вернуть ся 3. Ввести 3 — вернуть ся	Каждый пункт открывае т нужный модуль без ошибок	Успешно
18	Неверный пункт меню	Проверка обработки значения вне диапазона 1-4.	1. Ввести: 9	Сообщен ие «Неверн ый ввод», меню отобража ется снова	Успешно – ветка default в switch обрабатывает неверный ввод
19	Выход из программы	Проверка корректного завершения.	1. Ввести: 4	Сообщен ие «Програ мма завершен а!», програм ма закрывае тся	Успешно

5 Чек-листы

Чек-лист 1 — Smoke-тестирование

№	Проверка	Результат	Примечание
1	Приложение запускается без ошибок	+	Программа запускается через dotnet run без исключений
2	Отображается главное меню с 4 пунктами	+	Все 4 пункта отображаются корректно
3	Пункт 1 открывает кредитный калькулятор	+	Переход выполняется, отображается заголовок модуля
4	Пункт 2 открывает конвертер валют	+	Переход выполняется, отображаются курсы валют
5	Пункт 3 открывает калькулятор вкладов	+	Переход выполняется, отображается заголовок модуля
6	Пункт 4 завершает программу	+	Выводится сообщение «Программа завершена!»
7	Кредитный калькулятор выполняет расчёт без ошибок	+	Результат выводится при корректных входных данных
8	Конвертер валют выполняет конвертацию без ошибок	+	Результат выводится при корректных входных данных
9	Калькулятор вкладов выполняет расчёт без ошибок	+	Результат выводится для обоих типов вклада
10	После расчёта можно вернуться в главное меню	+	Нажатие любой клавиши возвращает в меню

Чек-лист 2 — Валидация ввода

№	Проверка	Результат	Примечание
1	Отрицательная сумма кредита — сообщение об ошибке	+	GetValidDoubleInput с minValue: 0 блокирует отрицательные значения
2	Нулевой срок кредита — сообщение об ошибке	+	GetValidIntInput с minValue: 1 не пропускает 0
3	Текст вместо числа в сумме кредита — сообщение об ошибке	+	double.TryParse возвращает false, выводится «Неверный формат»
4	Пустая строка в поле суммы — сообщение об ошибке	+	TryParse не принимает пустую строку, цикл повторяется
5	Отрицательная сумма вклада — сообщение об ошибке	+	Та же валидация через GetValidDoubleInput minValue: 0

6	Текст вместо числа в сроке вклада — сообщение об ошибке	+	int.TryParse отклоняет нечисловые значения
7	Дробное число в поле срока (12.5) — сообщение об ошибке	+	int.TryParse не принимает дробные числа
8	Несуществующая валюта GBP — сообщение об ошибке	+	Метод IsValidCurrency возвращает false, выбрасывается исключение
9	Пустая строка в поле валюты — сообщение об ошибке	+	IsValidCurrency не принимает пустую строку
10	Неверный тип вклада (например 5) — сообщение об ошибке	+	Ветка else выбрасывает исключение «Неверный выбор типа вклада»
11	Неверный пункт меню (например 9) — сообщение об ошибке	+	Ветка default в switch выводит сообщение и возвращает в меню
12	Отрицательная процентная ставка — сообщение об ошибке	+	GetValidDoubleInput с minValue: 0 блокирует отрицательные значения

Чек-лист 3 — Корректность расчётов

№	Проверка	Результат	Примечание
1	Кредит 100000 / 12 мес / 10% — платёж 8791.59 руб	+	Формула аннуитетного платежа реализована корректно
2	Кредит с нулевой ставкой — переплата равна 0	+	Отдельная ветка if (rate == 0) обрабатывает этот случай
3	Общая сумма выплат = ежемесячный платёж × срок	+	totalAmount = monthlyPayment * months
4	Переплата = общая сумма – сумма кредита	+	overpayment = totalAmount - amount
5	Конвертация 100 USD → 9000.00 RUB	+	Курс 90.0 применяется корректно
6	Конвертация 100 EUR → 9850.00 RUB	+	Курс 98.5 применяется корректно
7	Конвертация в ту же валюту — сумма не изменяется	+	Проверка if (from == to) return amount в начале метода
8	Конвертация EUR → USD через рубли даёт 109.44, а не 109.00	-	Баг: константа EURtoUSD = 1.09 объявлена в коде, но нигде не используется

9	Вклад без капитализации 100000 / 12 мес / 10% — доход 10000.00 руб	+	Формула $\text{Сумма} \times \text{Ставка} \times \text{Срок} / 12 / 100$ реализована корректно
10	Вклад с капитализацией 100000 / 12 мес / 10% — доход 10471.31 руб	+	Формула Math.Pow реализована корректно
11	Доход с капитализацией всегда больше дохода без капитализации	+	Математически подтверждено при одинаковых параметрах
12	Все результаты округляются до 2 знаков после запятой	+	Форматирование:F2 применяется ко всем выводимым значениям